Uniwersytet Radomski im. Kazimierza Pułaskiego w Radomiu	
Wydział Transportu, Elektrotechniki i Informatyki	
Techniczne zastosowania sieci neuronowych	
Rok akademicki 2025/2026	
Projekt: Implementacja sieci neuronowej do klasyfikacji znaków P, R, O	
Autorzy:	Ivan Pohodaev Andrii Yevlakh Maksymilian Wieczorek

1.Struktura sieci neuronowej

W projekcie wykorzystano wielowarstwową sieć neuronową typu MLP (Multilayer Perceptron), która składa się z dwóch warstw ukrytych. Architektura ta opiera się na 256 neuronach wejściowych, co odpowiada liczbie pikseli obrazu o wymiarach 16x16 rozłożonego do postaci wektora jednowymiarowego. Pierwsza warstwa ukryta h1 składa się z 12 neuronów, natomiast druga warstwa ukryta h2 zawiera 8 neuronów. W obu warstwach ukrytych zastosowano funkcję aktywacji Leaky ReLU, która pozwala na przepływ niewielkich wartości ujemnych i skutecznie zapobiega problemowi zamierających neuronów podczas procesu uczenia. Warstwa wyjściowa zawiera 3 neurony odpowiadające klasyfikowanym literom P, R oraz O, a zastosowana w niej funkcja Softmax pozwala na interpretację wyników w formie rozkładu prawdopodobieństwa. Wagi sieci nie są dobierane przypadkowo, lecz inicjalizowane losowo z wykorzystaniem metody opartej na odwrotności pierwiastka z liczby wejść danej warstwy, co zapewnia stabilną wariancję sygnału w całej strukturze i przyspiesza zbieżność algorytmu.[1]

2.Opis algorytmu uczenia

W projekcie wykorzystano algorytm uczenia nadzorowanego oparty na metodzie wstecznej propagacji błędów (czyli Backpropagation) z optymalizacją spadku gradientu. Proces ten polega na iteracyjnym dostosowywaniu macierzy wag **W1,W2** oraz **W3** w celu minimalizacji błędu między predykcją sieci a wartością zadaną.Jako funkcję aktywacji w warstwie ukrytejzastosowano Leaky ReLU, która dopuszcza niewielki przepływ sygnału dla wartości ujemnych (X * 0,01), co zapobiega całkowitemu wyłączaniu się neuronów podczas treningu. Na warstwie wyjściowej użyto funkcji Softmax, która interpretuje wyniki jako prawdopodobieństwo przynależności do jednej z trzech klas: P, R lub O.

Algorytm działa dynamicznie na danych dostarczanych przez użytkownika. Każdy wzorzec wprowadzony do bazy jest automatycznie powielany poprzez augmentację, co obejmuje:

- wykonanie serii obrotów o kąty -12°, -6°, 6° oraz 12°
- przesunięcia obrazu o jeden piksel w pionie i poziomie
- dodanie szumu Gaussa (wektor **n**)

Zabiegi te pozwalają uzyskać kilkanaście elementów uczących z jednego rysunku i uodparniają sieć na zniekształcenia. Trening odbywa się przez określoną liczbę epok (domyślnie 5), a w każdej z nich zbiór danych jest losowo mieszany, aby zapewnić lepszą zbieżność procesu optymalizacji.[2]

3.Ciąg uczący (CU)

Ciąg uczący został przygotowany w sposób półautomatyczny na podstawie ręcznie wprowadzonych wzorców liter P, R oraz O. Każdy pojedynczy szkic narysowany przez użytkownika na siatce o rozmiarze 16x16 pikseli (co daje 256 wejść do sieci) był poddawany procesowi augmentacji w celu sztucznego zwiększenia bazy danych. Wstępne przetwarzanie polega na wykadrowaniu rysunku do jego faktycznych krawędzi, przeskalowaniu do rozmiaru 12x12 pikseli i ponownym umieszczeniu w centrum matrycy 16x16. Dane wejściowe podlegają binaryzacji – piksele o natężeniu powyżej progu 0.3 przyjmują wartość 1.0, a pozostałe 0.0, po czym macierz jest spłaszczana do wektora jednowymiarowego. Dla każdego wprowadzonego zanku algorytm tworzy dodatkowe probki rotując obraz o kąt o : -12°, -6°,6° i 12°, o przesunięciu offset o jeden piksel w każdym z czterech kierunków głownych oraz nałożenie niwielkiego szumu losowego (szum Gaussa).Dzięki temu z jednego ręcznego rysunku uzyskiwano zestaw 10 elementów treningowych. Każdy element ciągu zawiera wartość zadaną w postaci etykiety liczbowej (0 dla P, 1 for R, 2 dla O), zakodowanej w systemie One-Hot podczas obliczeń błędu. [3,4]

4.Ciąg testowy (CT)

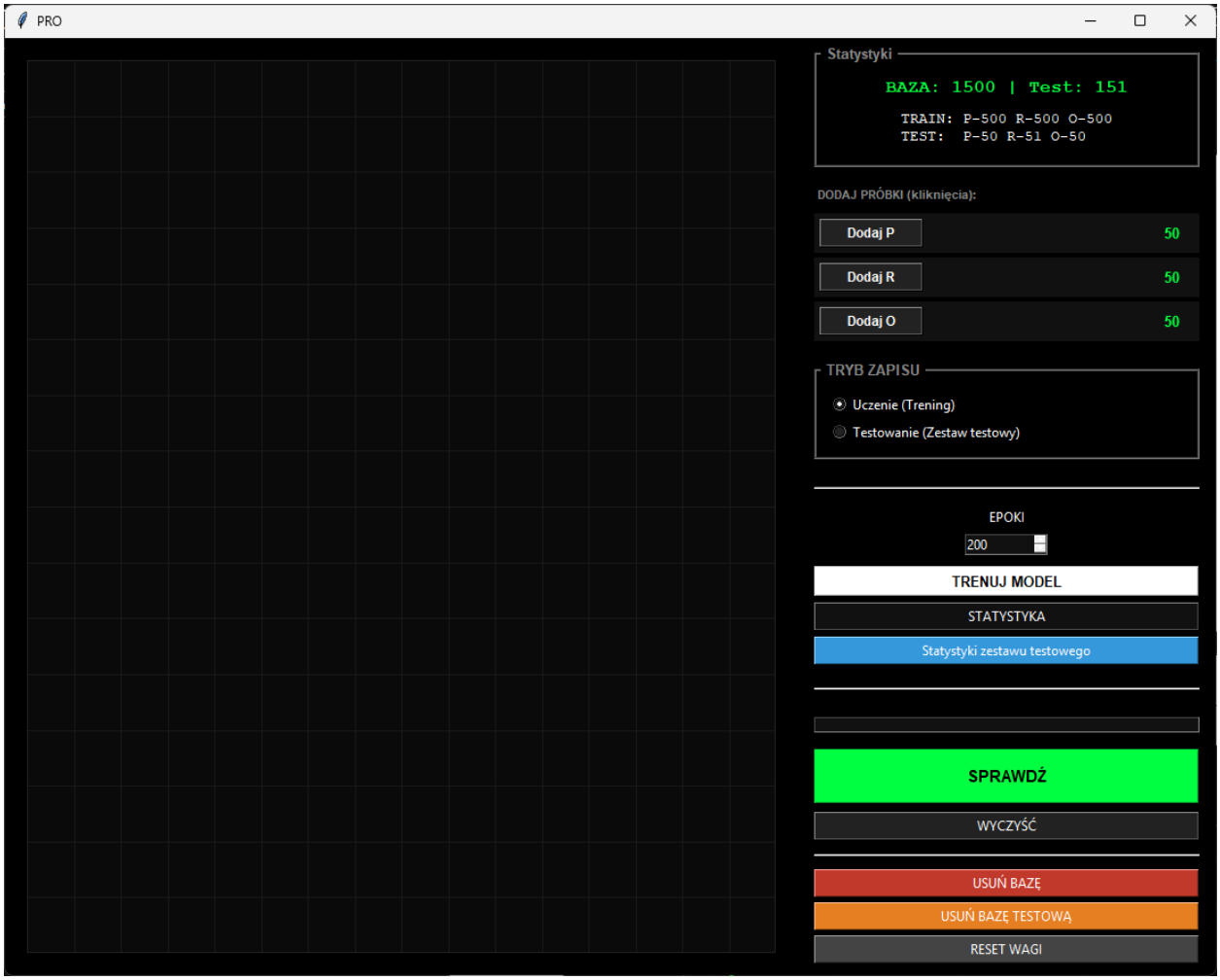
Ciąg testowy składa się z zestawu znaków wprowadzanych przez użytkownika w dedykowanym trybie testowym , które nie brały udziału w procesie optymalizacji wag (tzw treningu) . Podobnie jak w przypadku ciągu uczącego, dane testowe reprezentowane są przez wektory o długości 256 elementów, powstałe z przetworzenia rysunku o rozdzielczości 16x16 pikseli. Przed podaniem na wejście sieci, każdy znak testowy przechodzi identyczny proces przygotowania danych: jest wycinany do granic narysowanego kształtu, skalowany do formatu 12x12 i centrowany wewnątrz obszaru roboczego 16x16, a następnie poddawany binaryzacji z progiem 0,3. Pozwala to na uniezależnienie wyniku klasyfikacji od miejsca i wielkości narysowanej litery. Kluczową różnicą w stosunku do zbioru uczącego jest brak stosowania technik augmentacji – każda próbka w ciągu testowym jest unikalnym, pojedynczym wzorcem. Skuteczność na tym ciągu jest mierzona poprzez porównanie klasy wskazanej przez sieć (wyjście o najwyższym prawdopodobieństwie z funkcji Softmax) z etykietą nadaną przez użytkownika podczas zapisu. Pozwala to na obiektywną ocenę zdolności modelu do generalizacji i rozpoznawania nowych, nieznanych wcześniej przykładów pisma.[1,4]

5.Implementacja

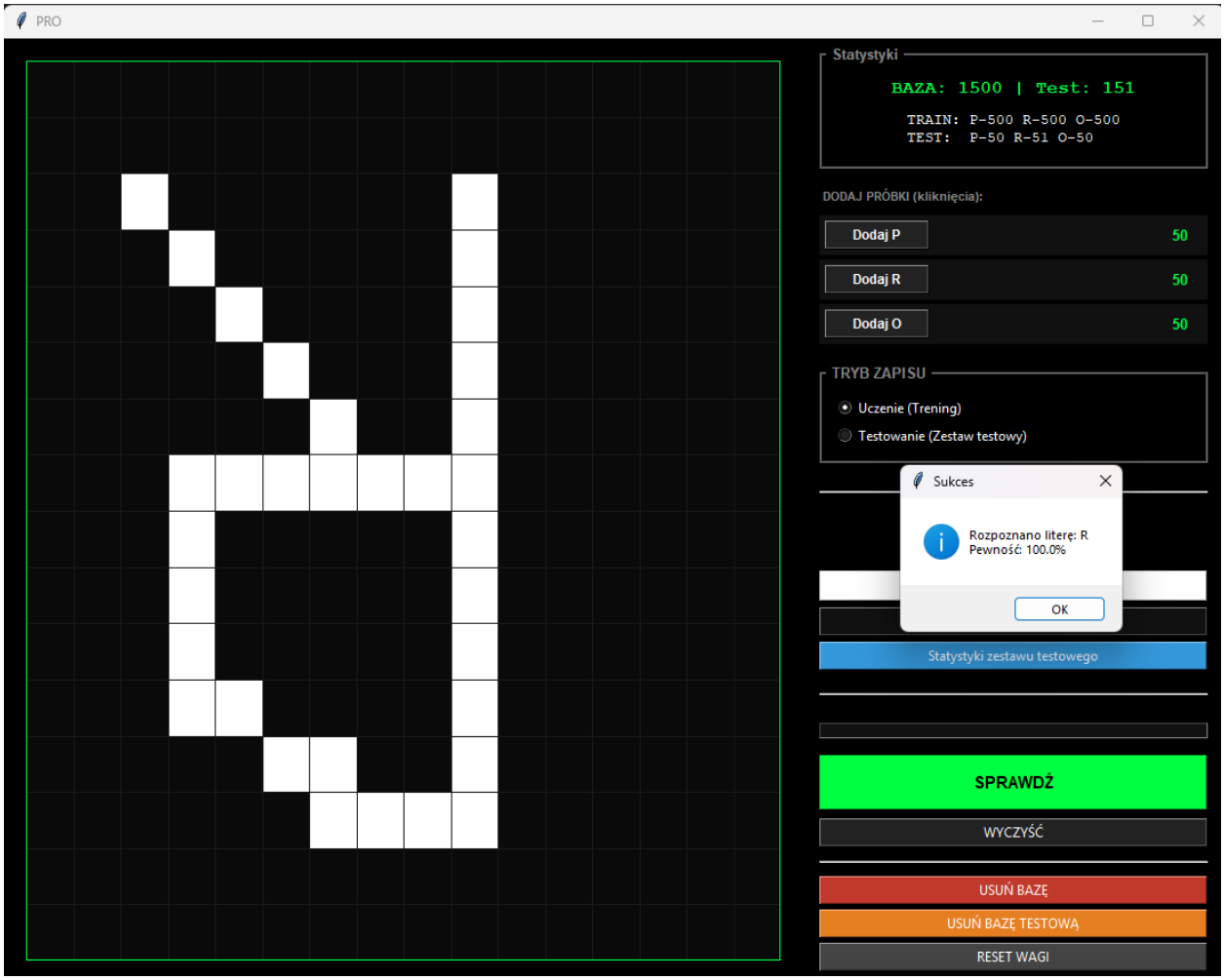
Aplikacja została przygotowana w języku Python z wykorzystaniem biblioteki Tkinter do stworzenia interfejsu graficznego oraz **Matplotlib** do generowania wykresów błędu i statystyk skuteczności. Obliczenia matematyczne oraz operacje na macierzach wag realizowane są przy pomocy biblioteki NumPy, natomiast do transformacji obrazów (rotacje, skalowanie) wykorzystano moduł **ndimage** z biblioteki SciPy.Interfejs składa się z interaktywnego pola rysowniczego o siatce 16x16, na którym użytkownik może kreślić znaki za pomocą myszy. Prawy panel boczny zawiera rozbudowane sekcje sterujące:

- Statystyki: sekcja ta pozwala na bieżące monitorowanie liczebności zbiorów. Po wciśnięciu przycisku „**Statystyki”**, program generuje wykres zbieżności błędu średniokwadratowego w czasie (funkcja kosztu), co pozwala ocenić postępy procesu uczenia [Rys. 4]. Z kolei przycisk „**Statystyki zestawu testowego”** wywołuje analizę skuteczności na danych nieznanych sieci, prezentując wyniki w formie wykresu słupkowego z podziałem na klasy P, R i O [Rys. 5].
- Dodawania: probek: dedykowane przyciski dla klas P, R, O, które uruchamiają procedurę augmentacji (rotacje, przesunięcia, szum).
- Trybu zapisu :przełącznik pozwalający decydować, czy nowe wzorce trafią do zbioru uczącego, czy testowego.
- Sekcji trenigowej umożliwia zdefiniowanie liczby epok i uruchomienie optymalizacji wag.

Obsługa programu jest intuicyjna – po zebraniu odpowiedniej liczby przykładów i przeprowadzeniu treningu, aplikacja staje się gotowa do pracy w trybie predykcji. Po naciśnięciu przycisku "SPRAWDŹ"[Rys.2], sieć dokonuje klasyfikacji, a wynik wraz z procentową pewnością wyświetlany jest w komunikacie systemowym. Stan sieci (czyli wagi) oraz zgromadzone dane są trwale zapisywane w plikach binarnych za pomocą modułu Pickle, co pozwala na kontynuację pracy z programem po jego ponownym uruchomieniu.[3,6]

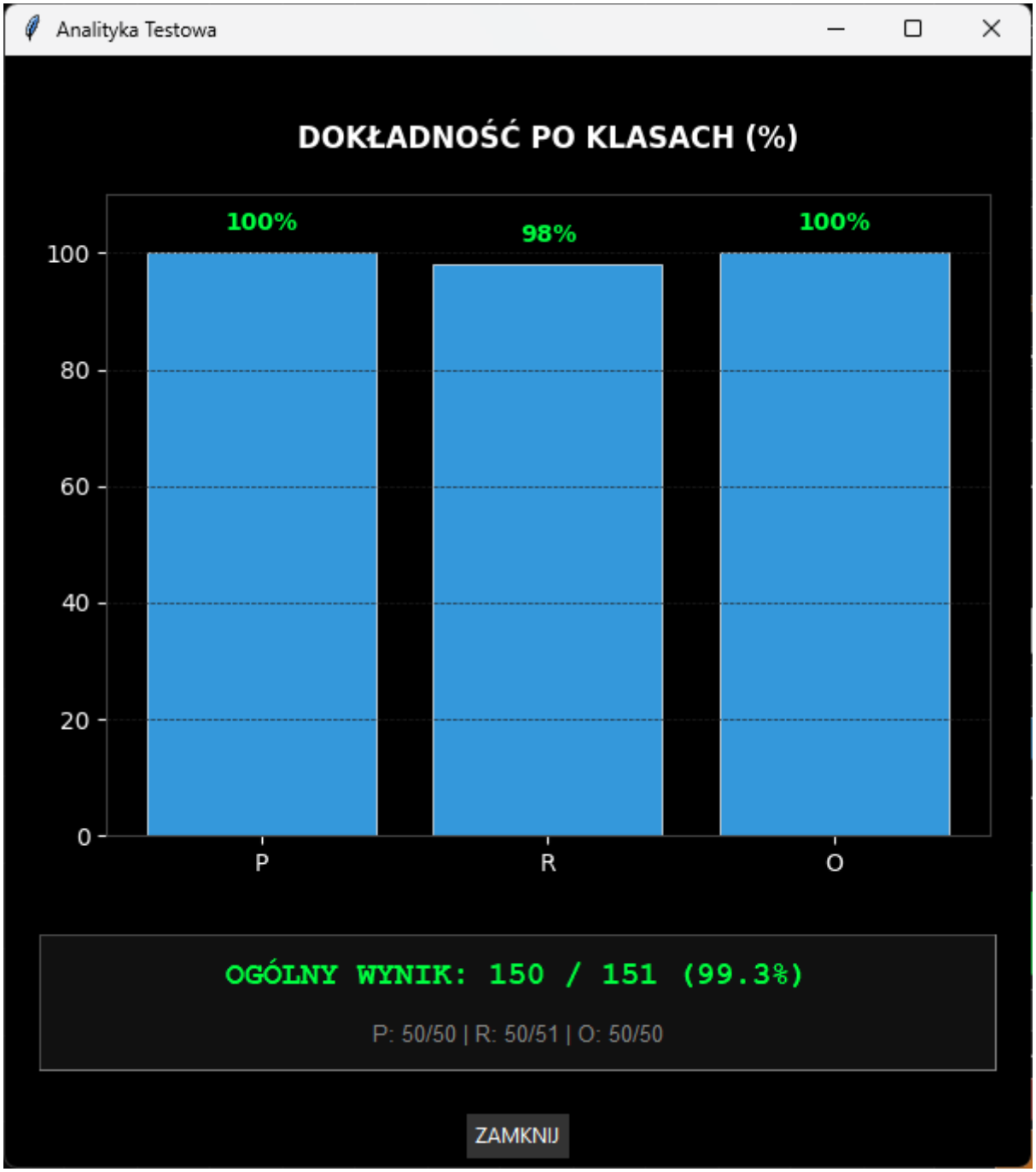


Rys. 1. Główne menu aplikacji z interfejsem graficznym
Źródło: Opracowanie własne



Rys. 2.Okno wyniku predykcji
Źródło: Opracowanie własne

Rys. 4. Wykres spadku błędu średnio kwadratowego (MSE) w funkcji epok
Źródło: Opracowanie własne



Rys.4 Statystyki dokładności
Źródło: Opracowanie własne

Instrukcja obsługi

1. Budowa bazy danych

- Rysowanie: Lewym przyciskiem myszy nakreśl znak na czarnym polu.
- Tryb zapisu: Wybierz Uczenie (do treningu) lub Testowanie (do późniejszej weryfikacji skuteczności).
- Dodawanie: Kliknij przycisk Dodaj P, R lub O. Program automatycznie wygeneruje 10 zróżnicowanych wariantów znaku (rotacje, przesunięcia, szum).

2. Trenowanie modelu

- Epoki: Wpisz liczbę cykli w polu EPOKI (zalecane 10–50).
- Start: Kliknij TRENUJ MODEL i czekaj na pasek postępu oraz komunikat końcowy.
- Wizualizacja: Przycisk STATYSTYKA wyświetla wykres spadku błędu w czasie.

3. Klasyfikacja i testy

• Rozpoznawanie: Narysuj znak i kliknij SPRAWDŹ. Wynik z procentową pewnością pojawi się w oknie.
• Weryfikacja: Przycisk Statystyki zestawu testowego pokazuje realną skuteczność modelu na nowych danych.

4. Narzędzia pomocnicze

• WYCZYŚĆ: Czyści pole rysownicze.
• USUŃ BAZĘ: Kasuje wszystkie zapisane wzorce z dysku.
• RESET WAGI: Przywraca losowe parametry sieci (zeruje proces nauki).

5.1. Opis najważniejszych fragmentów kodu

Najważniejszym elementem implementacji jest klasa DeepNeuralNetwork, która zawiera kompletną logikę sieci. Kluczowy fragment kodu to metoda train_step, realizująca proces uczenia:

Opis Rys.5: Powyższy fragment przedstawia implementację metody train_step, która odpowiada za proces uczenia sieci neuronowej poprzez algorytm wstecznej propagacji błędu. Funkcja wyznacza różnicę między przewidywaniem modelu a oczekiwanym wzorcem (target), a następnie oblicza gradienty wag dla obu warstw sieci z wykorzystaniem pochodnej funkcji aktywacji. Na tej podstawie dokonuje korekty wag **W1** , **W2** i **W3** przy użyciu współczynnika uczenia lr, dążąc do minimalizacji błędu.Całość procesu kończy się zwróceniem średniego błędu kwadratowego, co pozwala monitorować postępy treningu.[1,2]

```
49
50      def train_step(self, X, y_label):
51          target = np.zeros((1, 3))
52          target[0, y_label] = 1
53          output = self.forward(X)
54          error_out = output - target
55          d_W3 = self.a2.T.dot(error_out)
56          error_h2 = error_out.dot(self.W3.T) * self.leaky_relu_derivative(self.a2)
57          d_W2 = self.a1.T.dot(error_h2)
58          error_h1 = error_h2.dot(self.W2.T) * self.leaky_relu_derivative(self.a1)
59          d_W1 = self.a0.T.dot(error_h1)
60
61          self.W3 -= self.lr * d_W3
62          self.W2 -= self.lr * d_W2
63          self.W1 -= self.lr * d_W1
64
65          return np.mean(np.square(error_out))
```

Rys. 5. Algorytm uczenia
Źródło: Opracowanie własne

Opis Rys.6: Powyższy fragment kodu definiuje strukturę sieci neuronowej typu MLP . Parametry input_size (o wartości 256) odpowiadają spłaszczonej macierzy obrazu 16*16, a output_size (o wartości 3) reprezentuje klasyfikowane litery P, R i O. Sieć posiada dwie warstwy ukryte o rozmiarach kolejno 12 oraz 8 neuronów. Zastosowano tu inicjalizację wag metodą opartą na pierwiastku z odwrotności liczby wejść danej warstwy, co zapewnia stabilność sygnału i efektywny start procesu uczenia od pierwszej epoki. [3,6]

```python
    def __init__(self):
        self.input_size = 256
        self.h1_size = 12
        self.h2_size = 8
        self.output_size = 3
        self.lr = 0.01
        self.weights_file = "model_weights.pkl"
        self.history = []
        self.reset_weights()
        self.load_model()

    def reset_weights(self):
        self.W1 = np.random.randn(self.input_size, self.h1_size) * np.sqrt(1/self.input_size)
        self.W2 = np.random.randn(self.h1_size, self.h2_size) * np.sqrt(1/self.h1_size)
        self.W3 = np.random.randn(self.h2_size, self.output_size) * np.sqrt(1/self.h2_size)
        self.history = []
```

Rys. 6. Funkcje Aktywacji i Propagacja w przód
Źródło: Opracowanie własne

Opis Rys.7: Logika interfejsu graficznego opiera się na mapowaniu współrzędnych kursora myszy na indeksy dwuwymiarowej tablicy NumPy. Każdy 'piksel' narysowany przez użytkownika na komponencie Canvas jest zapisywany w macierzy jako wartość 1.0 . Metoda draw zapewnia natychmiastową informację zwrotną dla użytkownika, jednocześnie przygotowując dane binarne do dalszej obróbki przez sieć neuronową.[3,4]

```python
    def draw(self, event):
        self.canvas.config(highlightbackground="#222")

        w = self.canvas.winfo_width() / self.grid_size
        h = self.canvas.winfo_height() / self.grid_size
        x, y = int(event.x / w), int(event.y / h)
        if 0 <= x < self.grid_size and 0 <= y < self.grid_size:
            self.drawing_data[y, x] = 1.0
            self.canvas.create_rectangle(
                x * w, y * h, (x + 1) * w, (y + 1) * h, fill="white", outline="#1a1a1a"
            )

```

Rys. 7. Plansza do rysowania
Źródło: Opracowanie własne

Opis Rys.8: Ten fragment kodu realizuje proces augmentacji, czyli sztucznego powiększania zbioru treningowego. Z każdego pojedynczego rysunku system generuje kilkanaście dodatkowych próbek poprzez rotację o niewielkie kąty, przesunięcia o jeden piksel w każdym kierunku oraz nałożenie losowego szumu Gaussa. Zabieg ten drastycznie poprawia zdolność generalizacji modelu i zapobiega nadmiernemu dopasowaniu do idealnie narysowanych wzorców. [5]

```python
        for angle in [-12, -6, 6, 12]:
            rot = rotate(processed, angle, reshape=False, order=1)
            self.dataset.append((np.where(rot > 0.3, 1.0, 0.0).flatten(), label))

        for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            shifted = np.roll(processed, shift=dx, axis=1)
            shifted = np.roll(shifted, shift=dy, axis=0)
            self.dataset.append((shifted.flatten(), label))

        noise = np.random.normal(0, 0.05, processed.shape)
        noised = np.clip(processed + noise, 0, 1)
        self.dataset.append((noised.flatten(), label))
```

Rys. 8 . Augmentacja danych
Źródło: Opracowanie własne

Opis Rys.9: Metoda process_image odpowiada za normalizację danych wejściowych. Program automatycznie wykrywa krawędzie narysowanego znaku, wycina zbędne tło, a następnie skaluje literę do rozmiaru 12*12 pikseli przy użyciu interpolacji zoom. Dzięki osadzeniu tak przygotowanego znaku w stałej ramce 16*16, sieć staje się odporna na różnice w wielkości rysunku oraz jego położeniu na polu roboczym.[6,5]

```
332     def process_image(self, data):
333         rows = np.any(data, axis=1)
334         cols = np.any(data, axis=0)
335         if not np.any(rows):
336             return None
337         rmin, rmax = np.where(rows)[0][[0, -1]]
338         cmin, cmax = np.where(cols)[0][[0, -1]]
339         cropped = data[rmin : rmax + 1, cmin : cmax + 1]
340         try:
341             res = zoom(cropped, (12 / cropped.shape[0], 12 / cropped.shape[1]), order=0)
342             final = np.zeros((16, 16))
343             final[2:14, 2:14] = res
344             return np.where(final > 0.3, 1.0, 0.0)
345         except:
346             return data
```

Rys. 9. Przetwarzanie obrazu
Źródło: Opracowanie własne

Opis Rys.10: Do trwałego przechowywania parametrów modelu oraz zgromadzonej bazy danych wykorzystano moduł pickle. Dane są zapisywane w formacie binarnym (.pkl), co pozwala na zachowanie pełnej precyzji wag sieci oraz struktury obiektów bez konieczności ich konwersji na formaty tekstowe. Dzięki temu model może być trenowany wielokrotnie, a postępy nauki są automatycznie wczytywane przy każdym uruchomieniu aplikacji.[3]

```
60     def save_model(self):
61         with open(self.weights_file, "wb") as f:
62             pickle.dump({"W1": self.W1, "W2": self.W2, "history": self.history}, f)
```

Rys. 10.Zapis Binarny
Źródło: Opracowanie własne

Opis Rys.11: W warstwie ukrytej zastosowano funkcję aktywacji Leaky ReLU, która pozwala na przepływ minimalnych wartości ujemnych, co zapobiega całkowitemu wyłączaniu się neuronów podczas treningu. Warstwa wyjściowa korzysta z funkcji Softmax, która przekształca surowe wyniki sieci na wartości prawdopodobieństwa. Suma wyników dla wszystkich klas wynosi 1.0, co pozwala precyzyjnie określić pewność modelu podczas predykcji.[3]

```python
    def leaky_relu(self, x):
        return np.where(x > 0, x, x * 0.01)

    def leaky_relu_derivative(self, x):
        return np.where(x > 0, 1, 0.01)

    def softmax(self, x, T=1.0):
        ex = np.exp((x - np.max(x)) / T)
        return ex / np.sum(ex, axis=1, keepdims=True)

```

Rys. 11. Funkcja Aktywacyjna
Źródło: Opracowanie własne

6.Proces uczenia

Proces uczenia sieci neuronowej został przeprowadzony w trybie iteracyjnym, w którym podstawową jednostką miary czasu trwania sesji była epoka, definiowana jako jednokrotne przejście algorytmu przez cały dostępny zbiór danych treningowych. Liczba epok jest parametrem konfigurowalnym przez użytkownika w interfejsie graficznym aplikacji, co pozwala na elastyczne dostosowanie czasu treningu do bieżących potrzeb oraz wielkości zgromadzonej bazy próbek. Współczynnik uczenia został ustalony na stałym poziomie 0.01, co zapewnia stabilną aktualizację wag i minimalizuje ryzyko wystąpienia oscylacji wokół minimum funkcji straty.
W celu poprawy zdolności generalizacji modelu, przed rozpoczęciem każdej epoki zbiór treningowy jest poddawany losowemu mieszaniu, co zapobiega zapamiętywaniu przez sieć sekwencji wprowadzanych danych. Program na bieżąco monitoruje postępy procesu poprzez obliczanie średniego błędu kwadratowego dla każdej iteracji, a uzyskana historia strat jest archiwizowana w celu późniejszej wizualizacji na wykresie. Proces uczenia kończy się po wykonaniu zadanej liczby cykli, a finalne wartości wag są automatycznie zapisywane w pliku binarnym, co umożliwia ich późniejsze odtworzenie bez konieczności powtórnego trenowania modelu. Istotnym elementem wzmacniającym ten proces jest automatyczna augmentacja danych podczas dodawania próbek, dzięki której sieć uczy się rozpoznawać litery mimo ich przesunięć, rotacji czy obecności szumu cyfrowego.[2,4]

7.Testowanie

Proces testowania zrealizowano przy użyciu niezależnego zbioru danych zapisanego w osobnym pliku binarnym (test_dataset.pk1), co pozwoliło na obiektywną ocenę skuteczności modelu na danych, których nie widział on w fazie uczenia. System klasyfikacji poddano weryfikacji poprzez obliczenie ogólnej dokładności , zdefiniowanej jako stosunek liczby poprawnych wskazań do sumy wszystkich próbek testowych. Wyniki te zestawiono w formie graficznego wykresu słupkowego, który prezentuje skuteczność rozpoznawania z rozbiciem na poszczególne klasy liter P, R oraz O, co ułatwia identyfikację ewentualnych błędów wynikających z podobieństwa wizualnego znaków. Dodatkowym elementem procedury testowej jest wbudowany mechanizm kontroli pewności, który analizuje rozkład prawdopodobieństwa na wyjściu sieci; w przypadku, gdy różnica między dwiema najbardziej prawdopodobnymi klasami jest zbyt niska(ponizej 30) lub najwyższy(logit) wynik nie przekracza określonego progu (0,8) lub gęstość "atramentu" na rysunku jest nieprawidłowa (poniżej 0,05 lub powyżej 0,5), system zgłasza niepewność klasyfikacji. Takie podejście pozwala na precyzyjne określenie granic możliwości modelu oraz wpływu parametrów takich jak liczba epok czy jakość rysunku na ostateczną poprawność działania programu.[5,4]

8.Wnioski

Przeprowadzone badania nad działaniem zaimplementowanej sieci neuronowej typu MLP wykazały, że model skutecznie klasyfikuje odręcznie rysowane litery P, R oraz O, co potwierdza poprawność dobranej architektury oraz procesów przetwarzania danych. Kluczowym aspektem była realizacja przetwarzania wstępnego, gdzie automatyczne kadrowanie i skalowanie obrazu do formatu 12x12 pikseli w pełni ujednoliciło dane wejściowe, czyniąc klasyfikację niezależną od wielkości i położenia rysunku na kanwie. Równie istotna okazała się augmentacja zbioru treningowego poprzez rotacje w zakresie od -12° do 12°, przesunięcia oraz szum Gaussa, co dziesięciokrotnie zwiększyło bazę przykładów i poprawiło zdolność sieci do generalizacji nawet przy znacznym nachyleniu znaków. Dynamikę uczenia zoptymalizowano dzięki funkcji aktywacji Leaky ReLU, która zapobiegła zamieraniu neuronów, a w połączeniu ze współczynnikiem uczenia 0,01 zapewniła płynną minimalizację błędu średniokwadratowego bez niekorzystnych oscylacji. Dodatkowo wdrożony system kontroli pewności, analizujący progi prawdopodobieństwa i różnice między klasami, pozwolił na skuteczną identyfikację rysunków niejednoznacznych, co zapobiega błędnym klasyfikacjom przy danych niskiej jakości. Całość projektu, oparta na bezpośrednich obliczeniach macierzowych bez gotowych frameworków, potwierdziła swoją przydatność w rozpoznawaniu wzorców graficznych i stanowi solidny fundament do dalszego rozwoju aplikacji o kolejne klasy znaków.

9. Bibliografia

[1] Geron A., *Uczenie maszynowe z użyciem Scikit-Learn i TensorFlow*, Helion, Gliwice 2020.
[2] Goodfellow I., Bengio Y., Courville A., *Deep Learning. Systemy uczące się*, Wydawnictwo Naukowe PWN, Warszawa 2018.
[3] *NumPy Documentation*, https://numpy.org/doc/, data ostatniego dostępu: 26.03.2026.
[4] Rutkowski L., *Metody i techniki sztucznej inteligencji*, Wydawnictwo Naukowe PWN, Warszawa 2012.
[5] Tadeusiewicz R., *Sieci neuronowe*, Wydawnictwo Akademickie i Profesjonalne, Warszawa 2015.
[6] *Tkinter GUI library*, https://docs.python.org/3/library/tkinter.html, data ostatniego dostępu: 26.03.2026.

WKŁAD DO PROJEKTU		
Ivan Pohodaev	Andrii Yevlakh	Maksymilia Wieczorek
Wersja kodu • Pierwsza wersja • Trzeci wersja • Piąta wersja (**aktualna**) Robienie notatatek w sprawie kodu i sprawozdania	Wersje kodu • Droga wersja Edycja kodu o poprawki Edycja Sprawozdania Robienie notatatek w sprawie kodu i sprawozdania	Wersje kodu • Czwarta wersja Edycja kodu o poprawki Stworzenie i edycja sprawozdania Robienie notatatek w sprawie kodu i sprawozdania